In [ ]:
```
import marimo as mo
```

🔗 **Chats com LLM** (transcrições nesta pasta)

1. Discussão do enunciado: visão geral do problema
2. Geração da interface: figura, parâmetros e resultado

O "Closest Vector Problem" (CVP)

Seja um conjunto específico de vetores linearmente independentes $G \equiv \set{g_1, g_2, \ldots, g_k \mid g_i \in \reals^m}$; o reticulado determinado por G, $\mathcal{L}_G \equiv \set{x = z_1 g_1, z_2 g_2, \ldots, z_k g_k \mid z_i \in \Z}$. E finalmente um vetor aleatório t (target), com cada um de seus componentes >= 0, qual é o vetor $x \in \mathcal{L}_G$ que está mais próximo de (closest to) t?

In [ ]:
```
import heapq
import itertools
import random
import time
import warnings

import cvxpy as cp
import numpy as np

# Um nó com estado OPTIMAL_INACCURATE é tratado em branch_and_bound: não serve para podar
warnings.filterwarnings("ignore", message="Solution may be inaccurate")

def branch_and_bound(G, t, l, tempo_limite=60.0, tol=1e-6):
    inicio = time.perf_counter()
    k = G.shape[0]
    H = G @ G.T
    Gt = G @ t

    # Com Gᵀ = QR, ||Gᵀz − t||² = ||Rz − Qᵀt||² + resto²: o cone fica com k + 2 entradas em vez de m + 1
    Q, R = np.linalg.qr(G.T)
    alvo = Q.T @ t
    resto = np.linalg.norm(t - Q @ alvo)

    # Relaxação de cada nó: z contínuo com lo <= z <= hi e r >= ||Gᵀz − t||₂ (cone de segunda ordem)
    z, r = cp.Variable(k), cp.Variable()
    lo, hi = cp.Parameter(k), cp.Parameter(k)
    cone = cp.SOC(r, cp.hstack([alvo - R @ z, [resto]]))
    relaxacao = cp.Problem(cp.Minimize(r), [cone, z >= lo, z <= hi])

    # Passos de ±1 em uma ou duas coordenadas, para a pesquisa local
    passos = []
    for i in range(k):
```

```
        for si in (1, -1):
            passo = np.zeros(k)
            passo[i] = si
            passos.append(passo)
            for j in range(i + 1, k):
                for sj in (1, -1):
                    duplo = passo.copy()
                    duplo[j] = sj
                    passos.append(duplo)
    passos = np.array(passos)
    custos = (passos @ H * passos).sum(axis=1)

    # A partir de um z inteiro, dá sempre o passo que mais reduz dist², até
nenhum passo ajudar
    def pesquisa_local(z_int):
        gradiente = 2 * (H @ z_int - Gt)
        while True:
            variacao = passos @ gradiente + custos
            novos = z_int + passos
            variacao[((novos < 0) | (novos > l)).any(axis=1)] = np.inf
            melhor_passo = variacao.argmin()
            if variacao[melhor_passo] >= -1e-9:
                return z_int
            z_int = novos[melhor_passo]
            gradiente = gradiente + 2 * H @ passos[melhor_passo]

    melhor_z, melhor_d = np.zeros(k), np.linalg.norm(t)

    def avaliar(candidato):
        nonlocal melhor_z, melhor_d
        d = np.linalg.norm(G.T @ candidato - t)
        if d < melhor_d:
            melhor_z, melhor_d = candidato, d

    # Best-first: abre sempre o nó com menor limite inferior (o contador só
desempata)
    contador = itertools.count()
    abertos = [(0.0, next(contador), np.zeros(k), np.full(k, float(l)))]
    nos = 0
    while abertos and abertos[0][0] < melhor_d - tol and
time.perf_counter() - inicio < tempo_limite:
        limite, _, a, b = heapq.heappop(abertos)
        if (a == b).all():
            avaliar(a)
            continue

        lo.value, hi.value = a, b
        try:
            relaxacao.solve(solver=cp.CLARABEL)
            estado = relaxacao.status
        except cp.SolverError:
            estado = None
        nos += 1

        # Se o solver falhar, continua a ramificar a partir do meio da
caixa: nada fica por explorar
```

```
        v = z.value if estado in (cp.OPTIMAL, cp.OPTIMAL_INACCURATE) else
(a + b) / 2
        candidato = np.clip(np.round(v), 0, l)
        avaliar(pesquisa_local(candidato) if nos % 10 == 1 else candidato)

        # Só uma solução exata da relaxação dá um limite inferior em que se
possa confiar
        exato = estado == cp.OPTIMAL
        if exato:
            limite = max(limite, float(r.value))
        if limite >= melhor_d - tol:
            continue

        livres = np.flatnonzero(a < b)
        if exato:
            # Todo o z melhor que o atual cumpre (z − v)ᵀH(z − v) <
melhor_d² − r²:
            # a caixa que envolve esse elipsoide aperta os limites das
coordenadas livres
            inversa = np.diag(np.linalg.inv(H[np.ix_(livres, livres)]))
            raio = np.sqrt((melhor_d**2 - float(r.value) ** 2 + 1e-6) *
inversa)
            a, b = a.copy(), b.copy()
            a[livres] = np.maximum(a[livres], np.ceil(v[livres] - raio -
1e-6))
            b[livres] = np.minimum(b[livres], np.floor(v[livres] + raio +
1e-6))
            if (a > b).any():
                continue
            livres = np.flatnonzero(a < b)
            if livres.size == 0:
                avaliar(a)
                continue

        # Ramifica na coordenada mais fracionária: z_i <= corte ou z_i >=
corte + 1
        i = livres[np.abs(v[livres] - np.round(v[livres])).argmax()]
        corte = np.clip(np.floor(v[i]), a[i], b[i] - 1)
        b_baixo, a_cima = b.copy(), a.copy()
        b_baixo[i], a_cima[i] = corte, corte + 1
        heapq.heappush(abertos, (limite, next(contador), a, b_baixo))
        heapq.heappush(abertos, (limite, next(contador), a_cima, b))

    otimo = not abertos or abertos[0][0] >= melhor_d - tol
    return {
        "x": G.T @ melhor_z,
        "z": melhor_z.astype(int),
        "dist": float(melhor_d),
        "limite_inferior": float(melhor_d if otimo else abertos[0][0]),
        "otimo": otimo,
        "nos": nos,
        "tempo": time.perf_counter() - inicio,
    }

def solve_cvp(k, m, l, seed=None, tempo_limite=60.0):
    if any(v < 1 or v % 1 for v in (k, m, l)):
```

```
        raise ValueError("k, m e l têm de ser números inteiros positivos.")
    k, m, l = int(k), int(m), int(l)
    if k > m:
        raise ValueError("Reduz k ou aumenta m: em dimensão m não há mais
de m vetores linearmente independentes.")
    gerador = random.Random(seed)

    while True:
        G = []
        for i in range(0, k):
            g_i = []
            for j in range(0, m):
                g_ij = gerador.uniform(0, 1)
                g_i.append(g_ij)
            G.append(g_i)
        G = np.array(G)
        if np.linalg.matrix_rank(G) == k:
            break

    u = np.array([gerador.uniform(0, l) for _ in range(k)])
    t = G.T @ u
    return {**branch_and_bound(G, t, l, tempo_limite), "G": G, "t": t}
```



k · vetores em G

3

m · dimensão de cada vetor

5

l

10

Search for closest vector…